GUIs, JavaFX, & Event Driven Programming

F. Hamid

Sources

- https://openjfx.io/
- https://dev.java/learn/javafx/
- https://openjfx.io/javadoc/25/javafx.graphics/javafx/scene/Node.html
- https://www.geeksforgeeks.org/java/javafx-tutorial/

GUI: to arrange visual components and respond to user events.

- Java offers several toolkits for building **graphical user interfaces (GUIs)**:
  - **AWT — Abstract Window Toolkit:**
    - Java's original GUI toolkit.
    - It Provides windows, buttons, text fields, layouts, and event handling.
    - Many components use the operating system's native controls.
  - **Swing:**
    - Builds on AWT and provides a richer set of components, such as JButton, JTable, and JTextArea.
    - Most components are drawn by Java, allowing more customizable appearances.

GUI: to arrange visual components and respond to user events.

- Java offers several toolkits for building **graphical user interfaces (GUIs)**:
  - **JavaFX:** A newer toolkit with a scene graph, CSS styling, properties and bindings, animation, and optional FXML for defining interfaces. Uses classes such as Stage, Scene, and Button.

JavaFX

- AWT and Swing are included in the standard JDK; JavaFX is supplied separately.

- **JavaFX is best described as a GUI framework, delivered as a collection of libraries.**
  - Both terms can apply, but “framework” describes how it manages your application.

MAVEN

- **Maven is a build and dependency-management tool for Java projects.**
- It helps you:
  - **Download dependencies:** libraries your project needs, such as JavaFX.
  - **Compile code:** turn .java files into .class files.
  - **Run tests and package applications:** for example, create a JAR file.
- Maven reads **pom.xml** to learn how your project is configured.

pom.xml

- POM stands for **Project Object Model**.
- **pom.xml is** the configuration file for a Maven project.

- It tells Maven:
  - Your project's name and version.
  - Which libraries the project needs—for example, JavaFX.
  - Which Java version to use when compiling.
  - Which plugins to use for tasks such as running the application.

- The Java files contain our program; pom.xml tells Maven how to build and run it.

Project Creation & First App

Our class supplies the application's behavior; JavaFX manages its lifecycle.

```
public class LabelApp extends Application {


@Override
public void start(Stage stage) {

}


public static void main(String[] args) {
        launch(args);
}

}
```

Getting to know the code …

- **Application: The application class**
  - **`Application`** is the abstract base class for a **`JavaFX program`**.
- We extend it and **`override start()`** to create the window's controls, arrange them, and display the window.
- **LabelApp class:** Our application class, which **extends** the JavaFX Application class

Getting to know the code …

- **Stage: the window**

- A Stage represents a top-level window.
- JavaFX creates the primary stage and passes it to **`start()`.**
- An application **can create additional stages** for additional windows.

Getting to know the code …

- **Scene: the window's content**

- A Scene holds the hierarchy of visual elements, called the **scene graph**.

- A stage displays one scene at a time, but you can replace that scene.

Getting to know the code

- **Components and layout containers**

  - Controls are GUI elements such as labels, buttons, text fields, and dropdowns
  - Other controls include Button, TextField, TextArea, and ComboBox.

  - **Layout container >>**
    - Layout containers arrange these controls.
    - examples include StackPane, Vbox, HBox

Getting to know the code …

- launch() >> start JavaFX
  - launch() starts the JavaFX runtime and hands control to it.
  - JavaFX creates our application object and calls its lifecycle methods.
  - The command-line arguments are forwarded as application parameters.

The Application Lifecycle

Step	What happens
launch(args)	Starts JavaFX; JavaFX creates the application object, invoking its constructor.
init()	Performs initial setup. Optional to override.
start(Stage stage)	Builds and displays the interface. Required to override.
Application runs	JavaFX processes events and invokes registered handlers.
stop()	Performs cleanup during normal shutdown. Optional to override.

Overall Process

- We extend Application, put controls inside a layout, place that layout in a Scene, and display the scene in a Stage.

- Calling launch() starts JavaFX, which manages setup, user events, and shutdown.

To-Do 01

- Install Maven
- Setup your first project.
- Modify the text in the label.

To-Do 02 ...

- Create a new project on your own.
- Follow similar instructions as stated in the README.md
  - Create a new app **HelloJavaFX0** [source code provided]
  - pom.xml [make adjustments to the pom [mainclass]
    - <mainClass>demo.HelloJavaFX0</mainClass>
  - Run and show the outcome.

To-Do 03 ...

- In the same project you created HelloJavaFX0, create a new app **HelloJavaFX1**.
- Run it and notice the differences.
- Try to explain the setOnAction method.

```
greetButton.setOnAction(event -> {
    String name = nameField.getText().trim();

    if (name.isEmpty()) {
        greeting.setText(value: "Please enter your name.");
    } else {
        greeting.setText("Hello, " + name + "!");
    }

});
```

To-Do 03 [hints]

- greetButton.setOnAction(event -> { ....});

  - Registers an **EventHandler** using a lambda expression.
  - JavaFX runs the code inside the braces when the button is activated.

- Registering the handler does not execute its code immediately.
- The code runs each time the button is activated, using the text currently in the field.

Practice ...

- Check what HelloJavaFX2.java does.
- Try to modify a few things that you would like.

More next class …

Method	When the handler runs
setOnMouseClicked(…)	A mouse click occurs on the component
setOnMousePressed(…)	A mouse button is pressed
setOnMouseReleased(…)	A mouse button is released
setOnMouseEntered(…)	The pointer enters the component
setOnMouseExited(…)	The pointer leaves the component
setOnMouseMoved(…)	The pointer moves within the component
setOnKeyPressed(…)	A key is pressed while the component has keyboard focus
setOnKeyReleased(…)	A key is released
setOnKeyTyped(…)	Character input occurs
stage.setOnCloseRequest(…)	A request is made to close the window

More information ...

Container	Arrangement
StackPane	Children stacked in layers
VBox	Children arranged vertically
HBox	Children arranged horizontally
GridPane	Children arranged in rows and columns